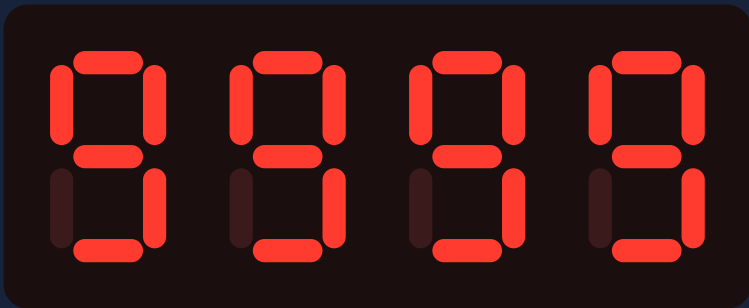


VERILOG · HOMEWORK

100 MHz 클럭 + 10 Hz Tick으로 만든 0 ~ 9999 카운터 · 4-Digit FND 표시

RTL 설계 · 합성 · 시뮬레이션 · Basys3 보드 검증
v1 Tick 카운터 → v2 스위치 제어 (Clear > Stop > Down)



참조 파일

vfile/source_file/1007_source_file/
vfile/constraint/FND_COU.xdc

정안성 · 2026. 10. 07

A 10 Hz Tick on One 100 MHz Clock Drives a 0–9999 Counter with Switch Priority

목표

보드 100 MHz 클럭 하나만으로 0.1초 Tick을 만들어 0~9999를 FND에 표시하고, 스위치 3개로 멈춤 · 초기화 · 감소를 제어한다

10 Hz

Tick 주기 0.1초

$100,000,000 \div 10,000,000$

14 bit

카운트 폭

$2^{14} = 16,384 \geq 10,000$

진행 결과

v1 Tick 생성기 + 카운터 → 0.1초마다 +1, 9999 다음 0

시뮬레이션 확인

v2 스위치 3개 추가, 8가지 조합을 표 하나로 판정

보드 확인

합성 100 MHz 타이밍 통과 — Setup 여유 4.652 ns

리포트 확인

v2 경계값 9999 ↔ 0 · 우선순위 전환 시뮬레이션

결과 공란

+4.652 ns

Setup 최소 여유

71개 경로 전부 통과

6 / 8

보드 확인 스위치 조합

000 · 001 단독 사진 없음

남은 일 — v2 경계값 시뮬레이션 결과 미첨부(p.20~21), 자동 판정 TB 없음.
검증 평가 19 / 40 (p.25)

개선 제안 — FND 스캔(1 kHz)은 아직 분주 클럭. Tick 방식으로 통일하면 설계 전체가 클럭 1개로 동작 (p.26)

v1 Counts on a Tick, v2 Adds Switch Priority, Next Step Is Boundary Verification

v1 Tick 카운터

- Tick 생성기: 1천만 클럭마다 1클럭 펄스
- 카운터: Tick마다 +1, 9999 다음 0
- FND 표시부: 기존 모듈 재사용
- 검증: 시뮬레이션 — Tick 주기, 끝값, 자리 분리



0.1초마다 +1 동작 확인

v2 스위치 제어

- 스위치 3개: 멈춤 · 초기화 · 감소
- 우선순위: 초기화 > 멈춤 > 감소
- 8가지 조합을 표 하나로 판정
- 검증: 보드 — 초기화 4조합, 멈춤, 감소



우선순위 보드 확인 · 시뮬레이션 공란

Next 검증 보강

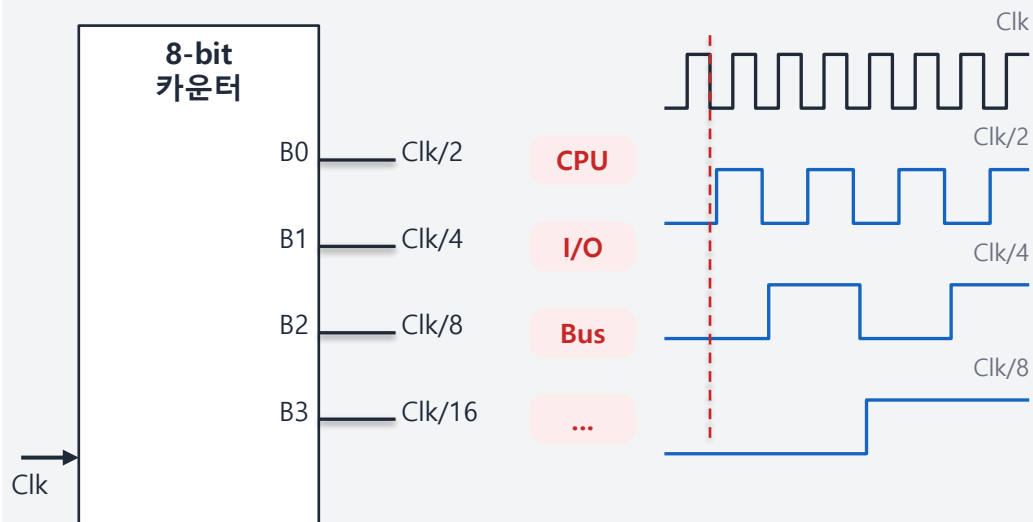
- Tick 주기를 줄인 시뮬레이션으로 경계값 확인
- 기대값과 자동 비교하는 TB
- FND 스캔도 Tick 방식으로 통일
- 스위치 입력 동기화

Tick 끝값을 10으로 줄이면
9999 경계까지 약 10만 클럭 (계산값)

제안 — p.26

Clock IPs Align Edges with a Dedicated Clock Tree — Not with Counter Bits

질문의 그림 — 카운터 비트를 클럭으로 나눠 쓰는 방식



문제

- 카운터 출력(Q)은 FF 지연 + 일반 배선을 거쳐 나감 → 비트마다 엣지가 조금씩 늦게 도착 (**Skew**)
- 도구가 이 신호를 클럭으로 관리하지 못하면 타이밍 검사에서 빠짐

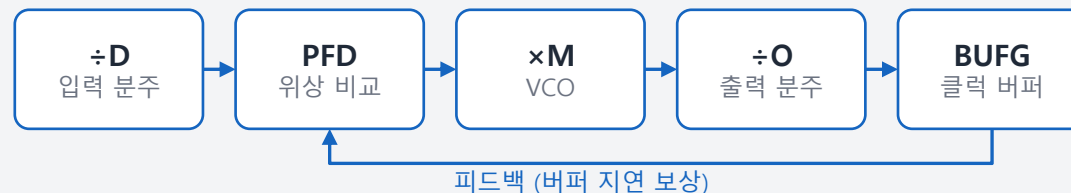
1 각 클럭의 타이밍을 어떻게 맞추나?

- 클럭 IP(MMCM/PLL)는 기준 클럭 하나로 고속 VCO를 만든 뒤 나눠서 출력 → 모든 출력이 같은 기준 엣지에서 나옴
- 피드백 경로로 클럭 버퍼 지연까지 측정해 입력 클럭과 위상을 맞춤 (Deskew)

2 Clock Skew는 어떻게 잡아내나?

- 물리: 전용 클럭 버퍼(BUFG) + 칩 전체에 길이를 맞춘 클럭 트리 → FF마다 도착 시간이 거의 같음
- 도구: 타이밍 분석이 FF마다 클럭 도착 시간을 계산해 Slack에 반영 → 리포트의 Clock Skew 항목으로 확인

3 내부 IP 로직은 어떻게 되어 있나?



$f_{out} = f_{in} \times M \div (D \times O)$ · 위상이 맞으면 LOCKED = 1 → 그때부터 사용

All 71 Timing Paths Pass at 100 MHz — the Tightest Path Still Has 4.652 ns to Spare

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.652 ns	Worst Hold Slack (WHS): 0.247 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 71	Total Number of Endpoints: 71	Total Number of Endpoints: 58

All user specified timing constraints are met.

+4.652 ns

Setup 최소 여유 (10 ns 주기 중)

단순 환산 시 약 187 MHz까지 여유 — 추정 ($1 \div (10 - 4.652) \text{ ns}$)

Setup — 너무 늦지 않은가

- 데이터가 다음 클럭 엣지 전에 도착하는지 검사 (최대 지연)
- **WNS 4.652 ns** — 가장 빠듯한 경로의 남은 시간
- **TNS 0** — 음수 여유의 합. 0이면 위반 없음
- **Failing 0 / 71** — 검사한 도착점 71개 모두 통과

Hold — 너무 빠르지 않은가

- 엣지 직후 데이터가 너무 빨리 바뀌어 직전 값을 덮지 않는지 검사 (최소 지연)
- **WHS 0.247 ns > 0** → 통과
- **THS 0 · Failing 0** — 위반 없음
- 클럭을 늦춰도 안 고쳐지는 위반이라 따로 봄

Pulse Width — 클럭 폭이 충분한가

- 클럭 High/Low 폭이 FF · 클럭 버퍼의 최소 요구 폭 이상인지 검사
- **WPWS 4.500 ns** — 5 ns 반주기 중 4.5 ns 여유
- **TPWS 0 · Failing 0 / 58** — 위반 없음

주의 — 1 kHz 분주 클럭으로 도는 FND 스캔 레지스터는 별도 클럭 제약이 없어 이 검사에서 빠졌을 가능성 → Check Timing(29) 항목 확인 필요 (추정)

Divided Clocks Split the Design into Clock Domains; a Tick Keeps Everything on One 100 MHz Clock



✗ 분주 클럭으로 카운터를 돌리면

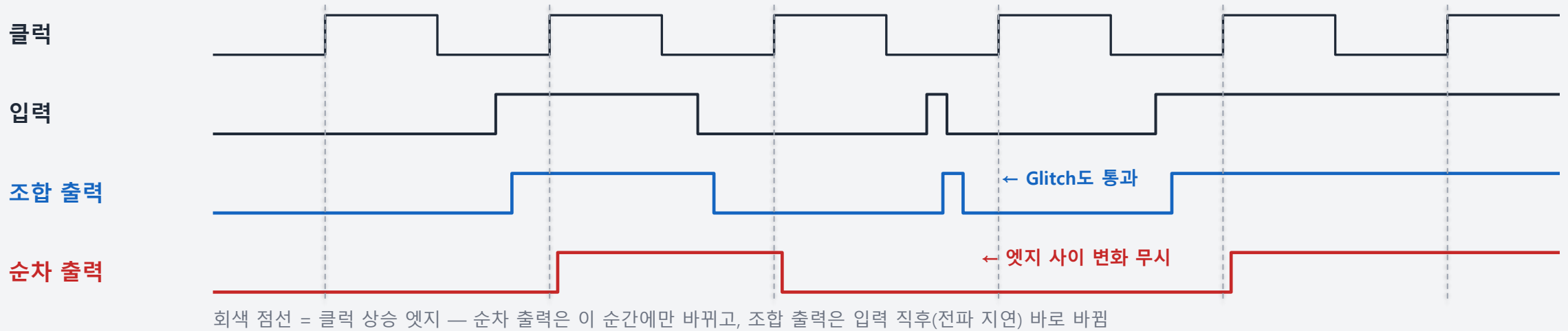
- 분주 출력은 일반 배선 → Skew가 큼 (Q1)
- 클럭 도메인이 늘어남 → 도메인 사이 신호 전달 시 메타스테이블 위험
- 도구가 새 클럭을 모르면 그 경로는 타이밍 검사 누락
- 조합 논리로 만든 클럭은 Glitch → 의도치 않은 엣지
- 전용 클럭 버퍼 수가 제한됨

✓ Tick을 쓰면

- 모든 FF가 100 MHz 클럭 1개 → 전 경로가 타이밍 검사 대상
- Tick = 1클럭 폭 '일 허가' 신호 → FF의 CE 핀으로 연결됨 (p.11)
- 100 MHz 자원은 그대로 쓰고, 일의 단위(0.1초)만 정의
- 주기는 숫자 하나(끝값)로 변경, 여러 주기 Tick도 같은 클럭에서

이 프로젝트 — 카운터는 Tick ✓ / FND 스캔은 아직 1 kHz 분주 클럭 ✗ → Tick으로 통일 제안 (p.26)

Combinational Output Follows the Input Immediately; Sequential Output Waits for the Clock Edge



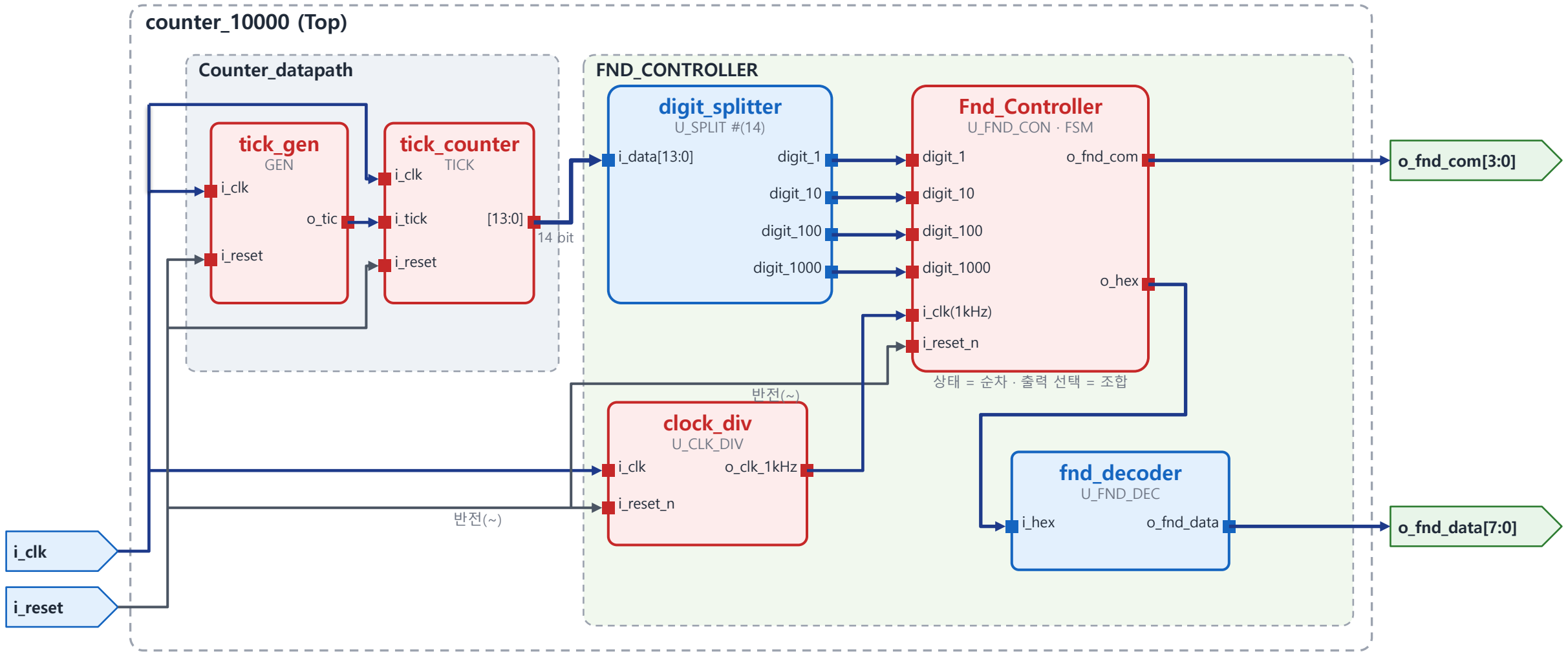
조합 회로 — 지금 입력만 본다

- 출력 = $f(\text{현재 입력})$ · 기억 없음 · 클럭 없음
- 입력이 바뀌면 전파 지연 후 즉시 출력 변화, 중간 Glitch 가능
- 이 설계: 자리 분리기, 7-세그먼트 디코더

순차 회로 — 입력 + 기억(상태)을 본다

- 출력 = $f(\text{입력, 저장된 상태})$ · FF가 값을 기억
- 클럭 엣지에서만 갱신, 그 사이는 값 유지 → 출력이 깨끗함
- 이 설계: Tick 생성기, 카운터, 분주기, FND 스캔 상태기

Top-Level Block Diagram — Sequential Blocks in Red, Combinational Blocks in Blue

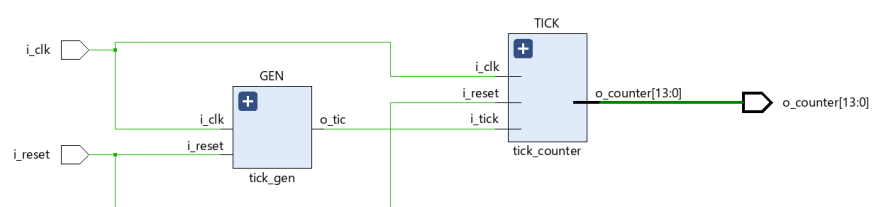


Module Roles — Tracing Count 1234 from the Tick to the Lit Segments



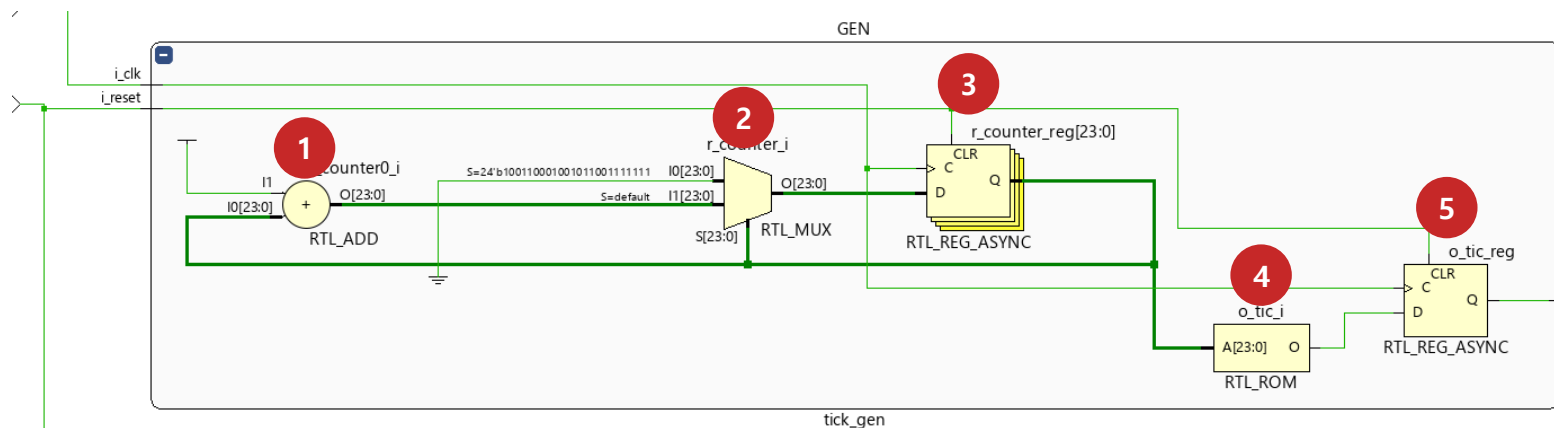
	모듈	종류	하는 일	왜 이렇게 만들었나
①	Tick 생성기 (tick_gen)	순차	0 ~ 9,999,999를 세고 끝값에서 1클럭 펄스	카운터는 100 MHz 그대로 두고 '0.1초 단위'만 만들기 위해
②	카운터 (tick_counter)	순차	Tick이 온 클럭에서만 +1, 9999 다음 0	14 bit(최대 16,383)가 9999를 담는 최소 폭
③	자리 분리기 (digit_splitter)	조합	2진 값 → 천·백·십·일의 자리 숫자	FND는 자리마다 0~9 숫자 하나를 받기 때문
④	분주기 (clock_div)	순차	100 MHz를 약 1 kHz 스캔 클럭으로	4자리를 빠르게 돌려 동시에 켜진 것처럼 보이게 (잔상)
⑤	FND 스캔 (Fnd_Controller)	순차	4개 상태 순환 — 상태값이 곧 자리 선택 신호	세그먼트 선을 4자리가 공유 → 한 번에 한 자리만 켜
⑥	7-세그 디코더 (fnd_decoder)	조합	숫자 0~9 → 세그먼트 8개 On/Off	공통 애노드 FND라 0일 때 켜짐 (Active-Low)

Inside the Tick Generator — Count 0 to 9,999,999, Then Emit One 10 ns Pulse



Datapath = Tick 생성기(GEN) → Tick → 카운터(TICK)

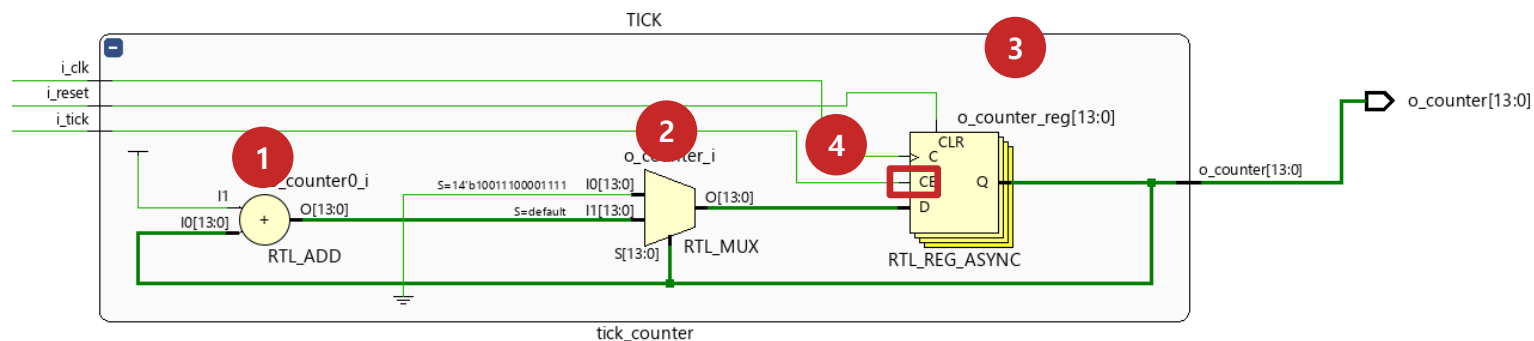
두 블록 모두 같은 100 MHz 클럭 · 같은 리셋 — 연결은 Tick 1가닥뿐. 아래는 GEN 내부 합성 결과



- 1 **덧셈기 (+1)**
현재 값 + 1을 매 클럭 계산
- 2 **선택기 (MUX)**
값이 끝값 9,999,999면 0, 아니면 +1 결과 → 끝에서 0으로 되돌림
- 3 **24 bit 레지스터**
 $2^{24} = 16,777,216 \geq 1\text{천만} \rightarrow 24\text{ bit}$. 리셋 버튼에 즉시 0 (비동기 CLR)
- 4 **비교기 (ROM 기호)**
'값 = 끝값?' 판단. 24 bit 입력 → 1 bit 출력 표라 도구가 ROM 기호로 그림
- 5 **Tick 레지스터**
비교 결과를 한 클럭 뒤 출력 → Glitch 없는 깨끗한 1 클럭(10 ns) 펄스

왜 이런 모양? — 코드의 '+1 / 끝값이면 0 / 끝값일 때 Tick = 1' 세 문장이 그대로 덧셈기 · 선택기 · 비교기로 바뀐 것

Inside the Counter — The Tick Lands on the Register's Clock-Enable Pin, Not on a Second Clock



- 1 덧셈기 (+1)
현재 값 + 1
- 2 선택기 (MUX)
값이 9999면 0 → 0~9999 순환
- 3 14 bit 레지스터
 $2^{14} = 16,384 \geq 10,000$. 리셋 시 즉시 0
- 4 CE 핀 = Tick
Tick이 1인 클럭에서만 새 값을 받아들임. 클럭은 100 MHz 그대로, '언제 일할지'만 Tick이 결정

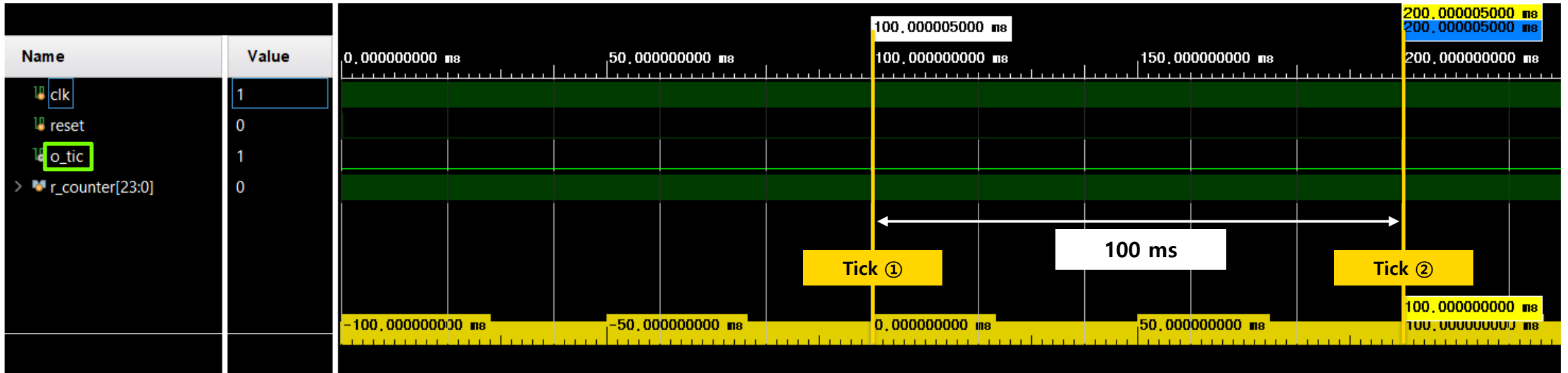
분주 클럭 방식이었다면

- 레지스터의 클럭(C) 핀에 분주 신호가 들어감
- → 새 클럭 도메인, 일반 배선 Skew, 타이밍 검사 누락 위험

Tick 방식 (이 설계)

- 클럭(C) 핀은 100 MHz 그대로, Tick은 CE 핀으로
- 코드에는 'Tick이 오면 +1'만 썼는데 합성기가 CE로 연결 — Q3 답의 증거

Tick Fires Every 100 ms — Exactly 10 Hz from the 100 MHz Clock



계산

- $100,000,000 \text{ Hz} \div 10 \text{ Hz} = 10,000,000$ 클럭마다 Tick 1번
- → 끝값 9,999,999

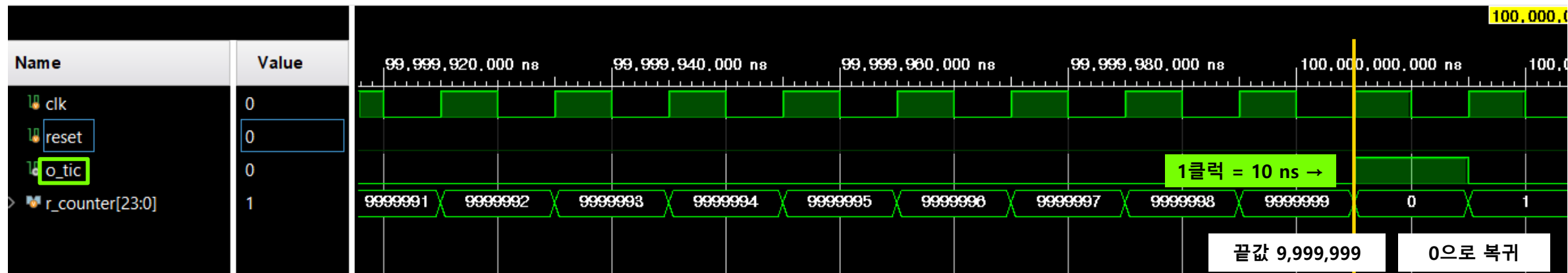
측정

- 커서 100.000005 ms → 200.000005 ms
- 간격 100.000000 ms = 정확히 10 Hz

+5 ns는 왜?

- 클럭 상승 엣지가 5, 15, 25 ns...에 있어 Tick도 엣지 시각에 맞춰 뜸
- 주기에는 영향 없음

At the Boundary, Count 9,999,999 Rolls to 0 and the Tick Is High for Exactly One Clock



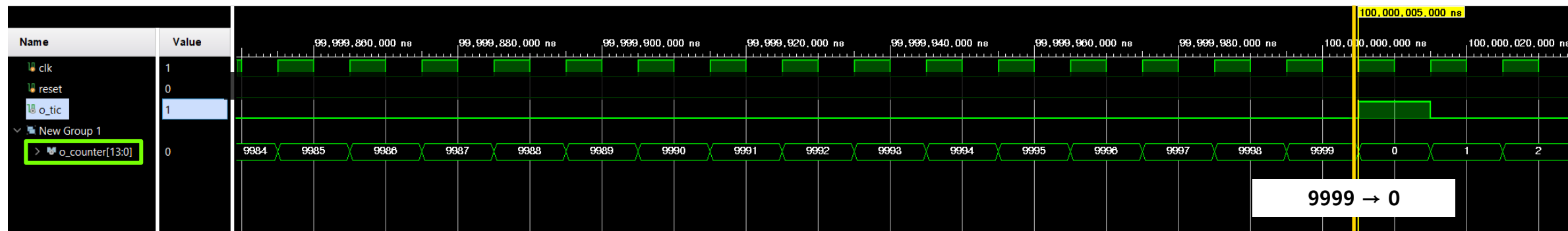
Tick이 끝값 '다음' 클럭에 뜨는 이유

- Tick도 레지스터 출력(p.10 ⑤) → 비교 결과가 한 클럭 늦게 나옴
- 카운트가 0이 되는 엣지와 Tick = 1이 되는 엣지가 같음

1클럭 폭이 중요한 이유

- 카운터는 Tick이 1인 클럭마다 +1 → 폭이 2클럭이면 +2가 됨
- 정확히 1클럭(10 ns) → Tick 1번에 카운터 +1 보장

Counter Boundary — After 9999 the Count Returns to 0



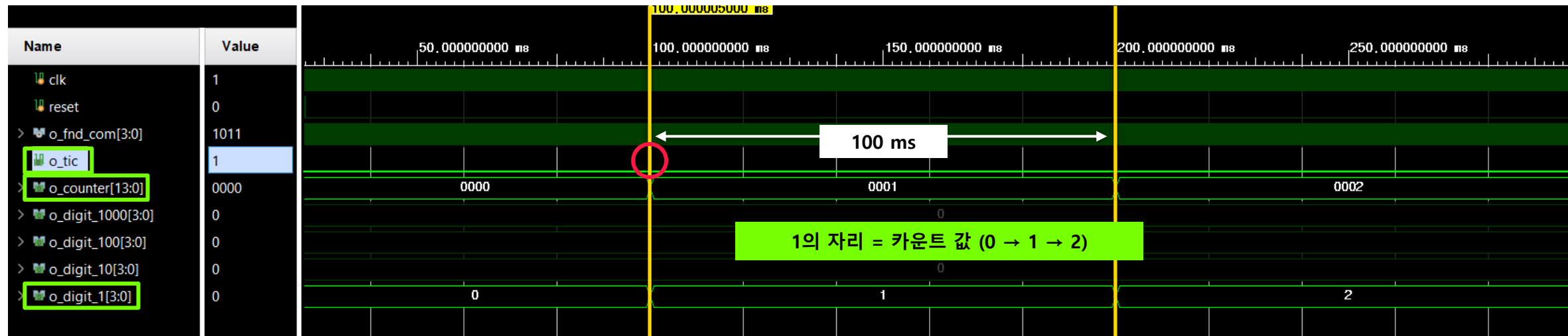
확인한 것

- 9999 다음 값이 10000이 아니라 0 → 4자리 범위 유지
- 끝값 비교(9999)와 0 되돌림 선택기(p.11 ②)가 의도대로 동작

캡처 해석 (참고)

- 이 캡처에서는 카운트가 10 ns마다 +1 → 매 클럭 증가 조건으로 끝값을 빠르게 확인한 것으로 보임
- Tick이 같은 시각에 뜬 것은 1천만이 1만의 배수라 겹친 것 — 실제 Tick 주기로 9999까지는 1,000초 (1천억 클럭, 계산값)

Linked Top Output — Each Tick Adds 1, and the Ones Digit Matches the Count



Tick ↑ → 카운트 +1

- Tick이 뜬 클럭에서 0000 → 0001 → 0002
- 100 ms 간격 = 0.1초마다 +1

자리 분리 즉시 반영

- 1의 자리가 카운트와 같은 시각에 바뀜
- 자리 분리는 조합 회로 → 클럭 대기 없음

초록 띠 = 자리 스캔

- FND 자리 선택 신호는 1 ms마다 바뀌어 이 배열에선 딱 찬 띠로 보임
- 어둡게 처리한 행

v2 Adds Three Switches — Clear Beats Stop, and Stop Beats Down

SW0 V17

멈춤 · STOP
현재 값 유지

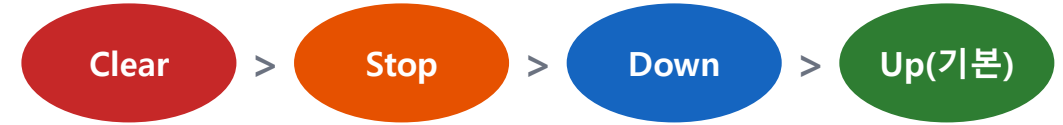
SW1 V16

초기화 · CLEAR
0으로

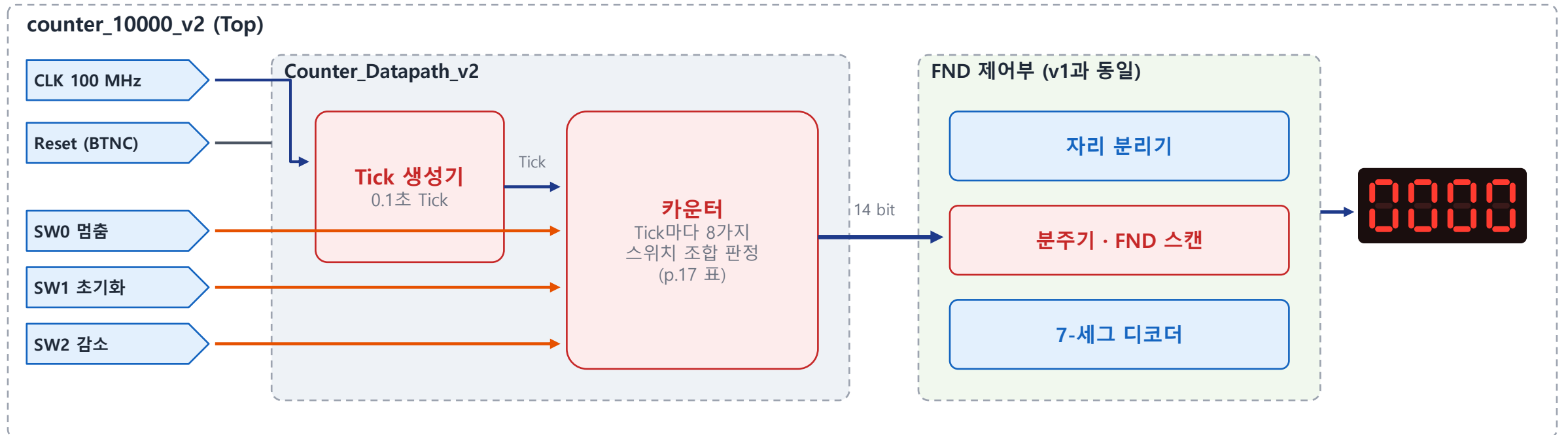
SW2 W16

감소 · DOWN
1씩 감소 (0 다음 9999)

우선순위



여러 스위치가 동시에 켜지면 왼쪽이 이김



Eight Switch Combinations Collapse into Four Actions — Priority Lives in One Lookup Table

SW2 감소	SW1 초기화	SW0 멈춤	동작	Tick 때 카운트	근거
0	0	0	증가	+1 (9999 다음 0)	아무것도 안 켜 → 기본 동작
0	0	1	멈춤	값 유지	멈춤만
0	1	0	초기화	0	초기화만
0	1	1	초기화	0	초기화 > 멈춤
1	0	0	감소	-1 (0 다음 9999)	감소만
1	0	1	멈춤	값 유지	멈춤 > 감소
1	1	0	초기화	0	초기화 > 감소
1	1	1	초기화	0	초기화 > 멈춤 > 감소

어떻게 구현했나

- Tick이 온 클럭에서만 스위치 3개를 묶어 8가지 경우를 표(case)로 판정
- 우선순위를 조건 순서가 아니라 경우별로 직접 적음 → 빠진 조합이 없는지 눈으로 확인 가능

4가지 동작, 8가지 입력

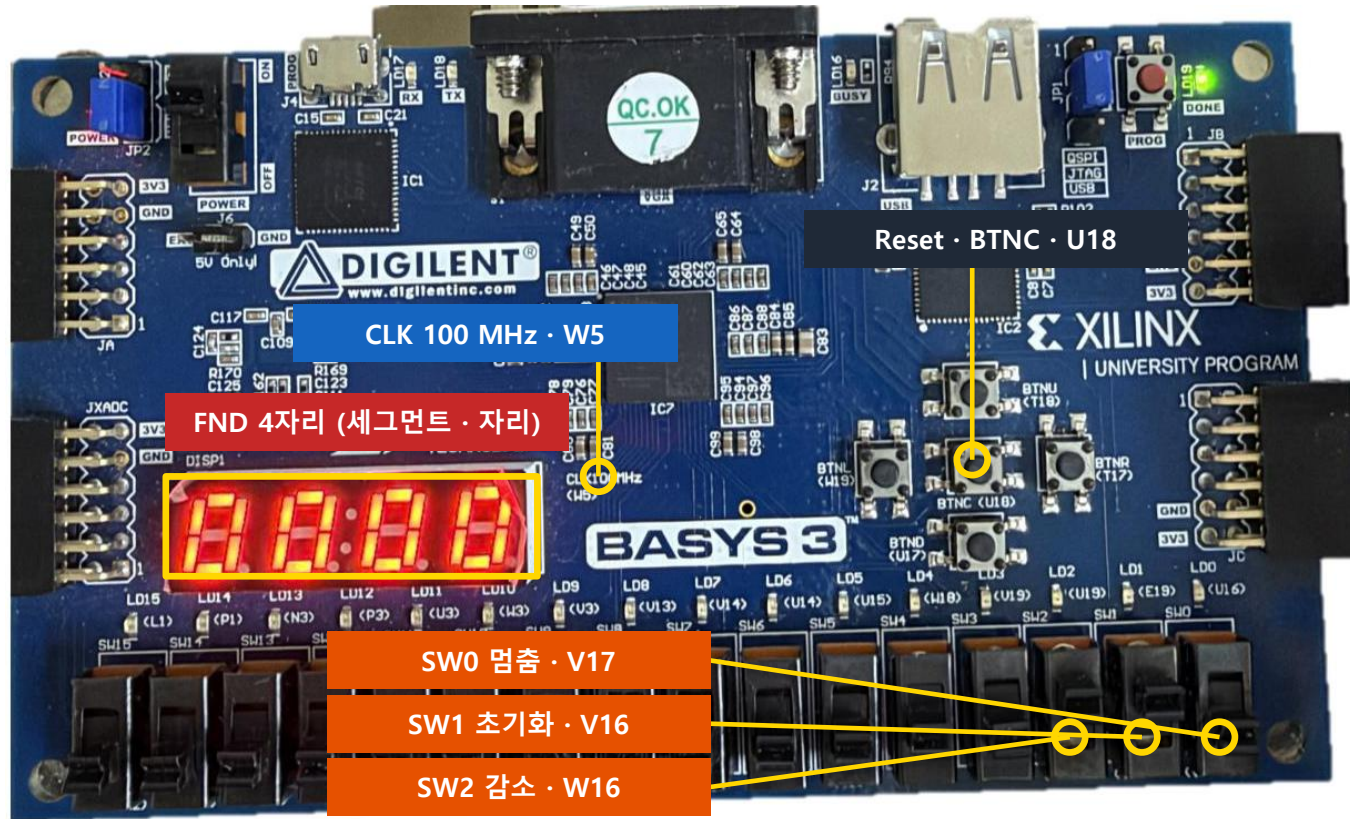
초기화 4 · 멈춤 2 · 감소 1 · 증가 1
초기화 스위치가 켜진 4줄은 모두 0

설계 특성 — 초기화도 Tick 때만 반영 → 스위치를 올린 뒤 최대 0.1초 후 0. 리셋 버튼(BTNC)은 즉시 0



18 / 27

Board I/O — Pin Map from FND_COU.xdc



신호	보드	핀	방향
i_clk	100 MHz 오실레이터	W5	입력
i_reset	BTNC (가운데 버튼)	U18	입력
i_sw0	SW0 — 멈춤	V17	입력
i_sw1	SW1 — 초기화	V16	입력
i_sw2	SW2 — 감소	W16	입력
o_fnd_data[0:7]	세그먼트 CA~CG, DP	W7 W6 U8 V8 U5 V5 U7 V7	출력
o_fnd_com[0:3]	자리 AN0~AN3	U2 U4 V4 W4	출력

클럭 제약

10 ns 주기(100 MHz) create_clock → 타이밍 분석의 기준 (p.5)
자리 선택 · 세그먼트 모두 0일 때 켜짐 (Active-Low)

Boundary Test Plan — Ten Cases to Check in Simulation (Results Pending)

#	확인 항목	입력 (SW2 SW1 SW0)	기대 결과	결과
1	증가 끝값	000	9998 → 9999 → 0	
2	감소 끝값	100	1 → 0 → 9999	
3	초기화	010	다음 Tick에 0 (최대 0.1초 지연)	
4	초기화 > 멈춤	011	0	
5	초기화 > 감소	110	0	
6	전부 켜	111	0	
7	멈춤	001	Tick이 와도 값 유지	
8	멈춤 > 감소	101	값 유지	
9	Tick 사이 스위치 변화	Tick 사이에 켜다 끄	변화 없음 (Tick 때만 판정)	
10	리셋	BTNC	즉시 0 (클럭 대기 없음)	

시뮬레이션 시간 줄이기

- 실제 Tick(1천만 클럭)으로 9999까지 가려면 1천억 클럭 — 계산값
- TB에서 Tick 끝값만 10으로 바꾸면 경계까지 약 10만 클럭(1 ms) — 계산값
- 설계 코드는 그대로, 숫자 하나만 TB에서 덮어씀

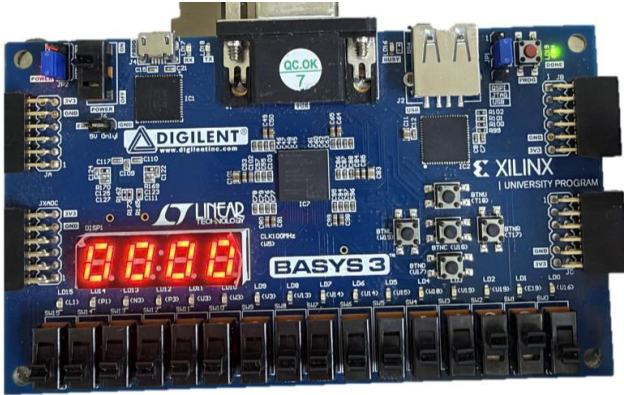
판정 방법

- 기대값을 설계와 따로 계산해 Tick마다 비교
- 불일치 수 · PASS/FAIL을 로그로 출력
- 결과 열은 시뮬레이션 후 채움

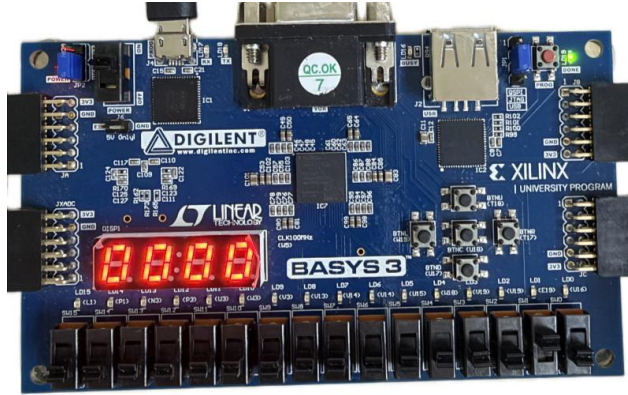
Simulation Result — Do Up, Down, Clear, and Stop Behave Correctly at the Boundary Values?

시뮬레이션 파형 캡처 자리
9999 → 0 · 0 → 9999 · 우선순위 전환 구간

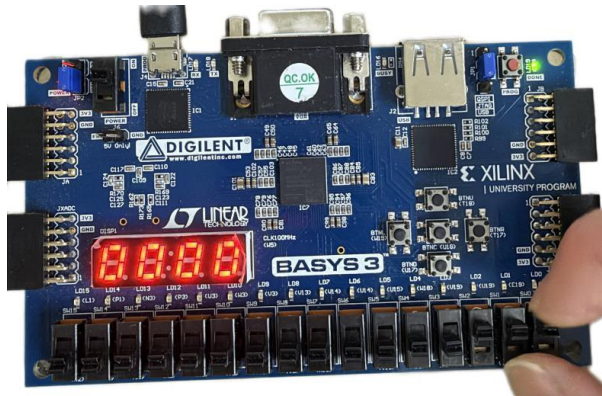
Board Result — Clear Wins in All Four Combinations and the Display Shows 0000



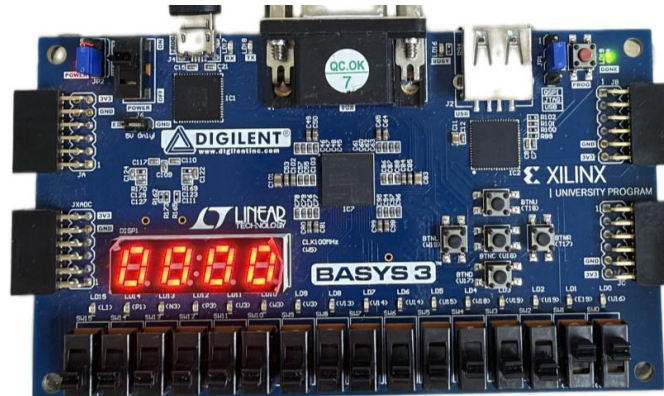
SW2 SW1 SW0 = 110 초기화 + 감소



SW2 SW1 SW0 = 010 초기화만



SW2 SW1 SW0 = 111 전부 컴



SW2 SW1 SW0 = 011 초기화 + 멈춤

판정

✓ 4가지 조합 모두 0000

초기화 스위치(SW1)가 켜지면 다른 스위치와 상관없이 0

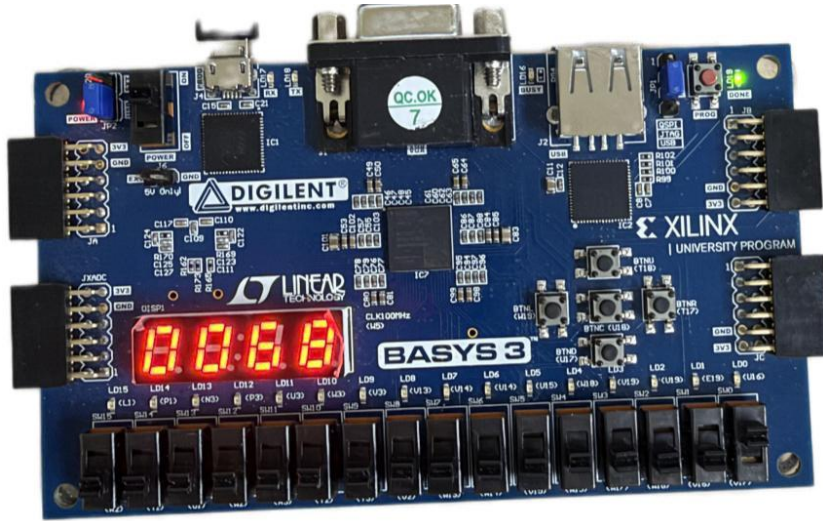
동작 표(p.17)의 초기화 4줄과 일치

한계

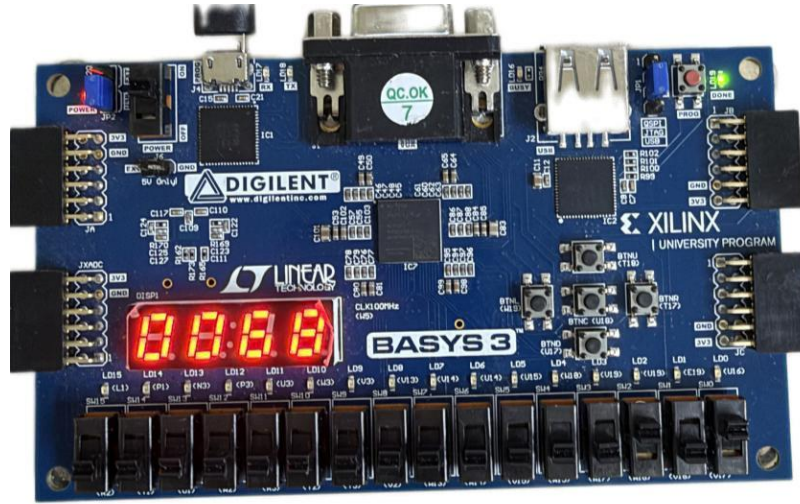
사진은 결과 화면 1장 — 0이 된 시점(최대 0.1초 지연)은 시뮬레이션에서 확인 (p.20 ③)



Board Result — Stop Holds the Count, and with Down Also On (101) Stop Still Wins



SW2 SW1 SW0 = 011



SW2 SW1 SW0 = 101

멈춤은 '쓰기 금지' — 레지스터 CE를 끄는 방식(p.18 ㉔)이라 Tick이 와도 값이 그대로

판정

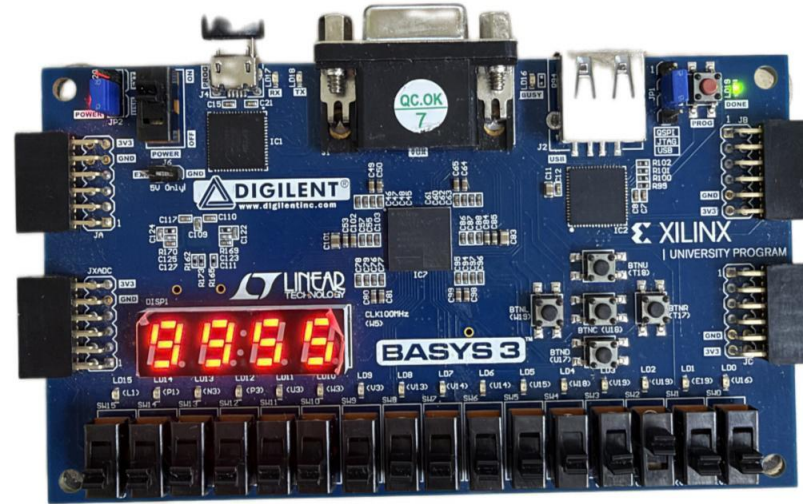
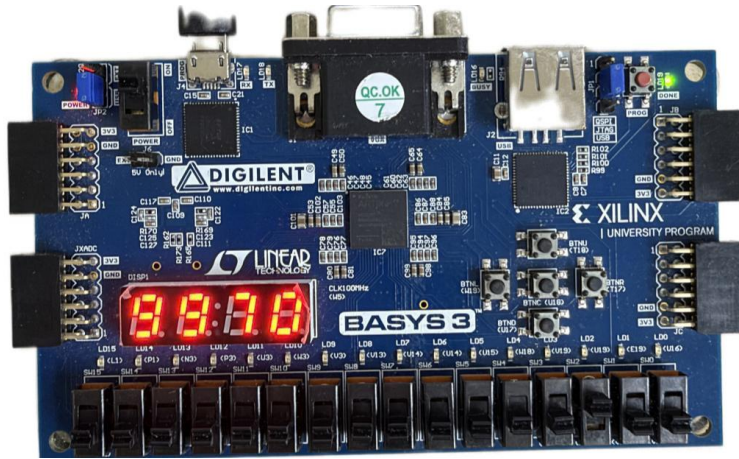
✓ 101: 멈춤 > 감소

멈춤과 감소가 함께 켜져도 값이 줄지 않고 유지

확인 필요

011은 동작 표상 초기화 조건(→ 0000). 사진 입력값 라벨 확인 후 수정

Board Result — Down Mode Counts Backward One Step per Tick



SW2만 켜 (100) — 감소 사진 2장 사이 표시값이 줄어듦 (99xx대)



0 → 9999
감소 경계 (기대 동작)

판정

✓ 감소 동작

Tick마다 1씩 감소

남은 확인

0 다음 9999 되돌림 순간은 사진으로 잡기 어려움 → 경계값 시뮬레이션(p.20 ②)

Verification Assessment 19 / 40 — Tick Timing Is Proven; Switch Logic Rests on Board Photos

항목	점수	근거
입력 커버리지	●●○○○	v1 Tick · 끝값 확인 / v2 스위치는 보드 사진 6/8 조합, 시뮬레이션 없음
기대값 독립성	●○○○○	기대값 없이 파형을 눈으로 비교
관측 지점	●●○○○	내부 카운트 · 자리 신호 관찰, 실제 FND 출력 핀 해석 없음
판정 타이밍	●●●●○	커서로 Tick 시각(100.000005 ms)과 1클럭 폭까지 정밀 확인
자동 판정 · 리포트	●○○○○	오류 수 · PASS/FAIL 출력 없음
보드용 Top 시뮬레이션	●●●○○	v1 Top 전체 시뮬레이션 ✓ / v2 Top 미실시
문제 해결 절차	●●●○○	우선순위를 조합별로 보드에서 재확인, 시뮬레이션 근거 없음
문서화 · 재현성	●●●○○	커서 시각 기록 양호 / TB가 실시간 3억 ns 대기로 느림

19 / 40
C · 보강 필요

가장 큰 빈칸

스위치 판정이 Tick 때만 일어나는 구조
— Tick과 스위치 변화가 겹치는 순간은
보드로 재현 불가

기준: v2 경계값 시뮬레이션 결과 첨부 전 · 결함
주입 미실시. 결과 추가 시 재평가

Next Steps — A Fast Self-Checking Testbench and a Single-Clock FND Scan

1 Tick 주기를 바꿀 수 있는 시뮬레이션

- TB에서 Tick 끝값만 10으로 덮어쓰기
- 9999 경계까지 1천억 → 약 10만 클럭 (계산값)
- p.20의 10가지 경우를 몇 ms 안에 확인

2 자동 판정 TB

- 입력 → 설계와 독립된 기대값 모델 → 비교 → 오류 수 · PASS/FAIL
- FND 출력 핀(세그먼트 · 자리)을 숫자로 되돌려 비교
- 사람 눈 대신 로그가 판정

3 FND 스캔도 Tick으로

- 1 kHz 분주 클럭 → 100 MHz 클럭 + 1 kHz Tick(CE)
- 설계 전체가 클럭 1개 → 모든 경로가 타이밍 검사 대상
- Q3 원칙(p.6)을 설계 전체에 일관 적용

4 스위치 입력 동기화

- 스위치는 클럭과 무관하게 바뀔 → FF 2단으로 받아 메타스테이블 방지
- 참고: 분주기는 100,001클럭 주기(≈ 999.99 Hz) — 동작 영향 없음

One Clock, Many Rates — Ticks Let 100 MHz Logic Work in 0.1-Second Steps

설계 결과

- 100 MHz 클럭 1개 + 0.1초 Tick으로 0~9999 카운터 완성
- 스위치 8조합을 표 하나로 판정 — 초기화 > 멈춤 > 감소
- 합성 결과에서 Tick이 레지스터 CE 핀으로 연결됨을 확인

검증 현황

- Tick 주기 100 ms, 1클럭 폭, 9999 → 0 시뮬레이션 확인
- v2 우선순위는 보드 사진으로 확인 (6/8 조합)
- v2 경계값 시뮬레이션 결과 공란 — 평가 19 / 40

배운 점

- 클럭을 나누지 말고 '일할 때'를 나눈다 — Tick = Enable
- 하드웨어는 모든 경우를 병렬로 만들고 고르기만 한다
- 긴 주기 설계는 TB에서 주기를 줄여야 경계값을 볼 수 있다